

Unit 03

Normalization

Outlines

Functional dependencies,

Normal forms, first, second, third normal forms, BCNF, inclusion dependence, lossless join decompositions, normalization using FD.

Coming up with a ‘good’ schema is very important

Scope of Normalization

1. How do we characterize the “**goodness**” of a schema ?
2. If two or more alternative schemas are available how do we compare them ?
3. What are the problems with “bad” schema designs ?
4. **Normal Forms:** Each normal form specifies certain conditions
If the conditions are satisfied by the schema certain kind of problems are avoided

Scenario : Student Dept Schema

"In a university database, we need to store details of students and their respective departments. Considering the concepts of database normalization and redundancy, how should we design the schema to ensure efficient storage and data integrity?"

1. What attributes should be included in the **Student** and **Department** tables?
2. How should we establish a relationship between these tables?
3. What issues can arise if we store department details directly within the student table?
4. How does normalization help in this scenario?
5. Based on the given schemas (correct and incorrect), which design would you choose and why?"

Example..

Correct schema:

Student

studName	<u>rollNo</u>	sex	studDept
----------	---------------	-----	----------

Department

<u>deptName</u>	officePhone	HOD
-----------------	-------------	-----

Incorrect schema:

Student Dept

studName	rollNo	sex	deptName	officePhone	HOD
----------	--------	-----	----------	-------------	-----

What will be the problem arises in incorrect schema ?

Brief explanation of previous example

Instance of Incorrect Schema:

studName	rollNo	sex	deptName	officePhone	HOD
Alice	101	F	CS	12345	Dr. Smith
Bob	102	M	IT	67890	Dr. John
Charlie	103	M	CS	12345	Dr. Smith

✗ Problems:

1. **Redundancy:** CS department details are repeated for Alice and Charlie.
2. **Update Anomalies:** If the **HOD** or **officePhone** for CS changes, it needs to be updated in multiple rows, increasing the risk of errors.
3. **Insertion Anomalies:** If a new department is created but has no students yet, we cannot insert it into the table.
4. **Deletion Anomalies:** If the last student of the **CS department** is deleted, department information is also lost.

Solution

To avoid these issues, we should **normalize the schema into two separate tables**.

Brief explanation of previous example

Student Table

studName	rollNo	sex	studDept
Alice	101	F	CS
Bob	102	M	IT

Department Table

deptName	officePhone	HOD
CS	12345	Dr. Smith
IT	67890	Dr. John

✓ Advantages:

1. **No redundancy:** Department details are stored only once.
2. **Consistency:** If the office phone or HOD changes, it updates in one place.
3. **Efficient storage and querying.**

Problems with Bad Database Schema

1. Redundant Data Storage

- **Example:** Office Phone & HOD information
 - Stored multiple times for each student in a department.
- **Implications:**
 - **Wasted Disk Space:** Unnecessarily increases storage costs.
 - **Increased Maintenance:** Updates must be made in several locations.
 - minimize chances of errors during updates.

2. Challenges of Updating Data

- When a program updates the office phone:
 - Must change it in multiple records.
- **Consequences:**
 - **Longer Execution Time:** More places to update leads to inefficiencies.
 - **Higher Error Risk:** Increased chances of inconsistency and mistakes.

3. Importance of Transaction Efficiency

- Transactions should complete as quickly as possible to ensure:
 - **System Performance:** Reduces wait times for users.
 - **Data Integrity:** Minimize the chances during updates

Problem with bad Schema

1. Insertion Anomaly
2. Deletion Anomaly
3. Update Anomaly

Anomalies in Bad Database Schema

Insertion Anomaly

- Issue: Cannot add a new department without entering a dummy student record.
- Implication: Forces irrelevant data into the database, leading to unnecessary complexity.

2. Deletion Anomaly

- Issue: Deleting all students from a department results in loss of department information.
- Implication: Critical data about the department may be unintentionally removed, complicating data retrieval.

3. Update Anomaly

- Issue: Updating the office phone number for a department requires changes in multiple records.
 - If any record is overlooked:
 - Consequence: Leads to data inconsistency, compromising data integrity.

Normal Forms

First Normal Form **(1NF)** - included in the definition of a relation

Second Normal Form **(2NF)**

Third Normal Form **(3NF)**

Boyce-Codd Normal Form **(BCNF)**

Fourth Normal Form **(4NF)** - defined using multivalued dependencies

Functional Dependencies

A functional dependency (FD) $X \rightarrow Y$
(read as X *determines* Y) ($X \subseteq R, Y \subseteq R$)
is said to hold on a schema R if
in any instance r on R ,
if two tuples t_1, t_2 ($t_1 \neq t_2, t_1 \in r, t_2 \in r$)
agree on X i.e. $t_1[X] = t_2[X]$
then they also agree on Y i.e. $t_1[Y] = t_2[Y]$

Note: If $K \subset R$ is a key for R then for any $A \in R$,
 $K \rightarrow A$
holds because the above ifthen condition is
vacuously true

Functional Dependency – Examples

Consider the schema:

Student (studName, rollNo, gender, dept, hostelName, roomNo)

Since rollNo is a key, $\text{rollNo} \rightarrow \{\text{studName, gender, dept, hostelName, roomNo}\}$

Suppose that each student is given a hostel room exclusively, then $\text{hostelName, roomNo} \rightarrow \text{rollNo}$

Suppose boys and girls are accommodated in separate hostels, then $\text{hostelName} \rightarrow \text{gender}$

Trivial and Non Trivial FD's

An **FD** $X \rightarrow Y$ where $Y \subseteq X$ - called a trivial FD, it always holds good

An **FD** $X \rightarrow Y$ where $Y \not\subseteq X$ - non-trivial FD

An **FD** $X \rightarrow Y$ where $X \cap Y = \text{NULL}$ - completely non-trivial FD

Deriving new FDs

Given that a set of FDs F holds on R we can infer that a certain new FD must also hold on R

For instance, given that $X \rightarrow Y$, $Y \rightarrow Z$ hold on R we can infer that $X \rightarrow Z$ must also hold

How to systematically obtain all such new FDs ?

Unless all FDs are known, a relation schema is not fully specified

Entailment Relation

We say that a set of FDs $F \models \{ X \rightarrow Y \}$

(read as F *entails* $X \rightarrow Y$ or

F *logically implies* $X \rightarrow Y$)

if in every instance r of R on which FDs F hold,
FD $X \rightarrow Y$ also holds.

Armstrong came up with several inference rules
for deriving new FDs from a given set of FDs

We define $F^+ = \{ X \rightarrow Y \mid F \models X \rightarrow Y \}$

F^+ : Closure of F

Practice Question

- 1) Given below is the table `department_manager_employee`

department_manager_employee		
employee_id	department	manager
1101	Finance	Sam
1102	R&D	Rohit
1101	HR	Sam
1103	Retail	Rohit

Find out the functional dependencies that **do not** hold on the above table:

- a) $employee_id \rightarrow manager$
- b) $employee_id \rightarrow department$
- c) $manager \rightarrow department$
- d) $department \rightarrow manager$

1) Given below is the table `department_manager_employee`

department_manager_employee		
employee_id	department	manager
1101	Finance	Sam
1102	R&D	Rohit
1101	HR	Sam
1103	Retail	Rohit

Find out the functional dependencies that **do not** hold on the above table:

- a) $employee_id \rightarrow manager$ **True**
- b) $employee_id \rightarrow department$ **False**
- c) $manager \rightarrow department$ **False**
- d) $department \rightarrow manager$ **True**

More practice questions: **Self Practices**
[click here for document](#)

Armstrong's Inference Rules (1/2)

1. Reflexive rule

$F \models \{X \rightarrow Y \mid Y \subseteq X\}$ for any X . Trivial FDs

2. Augmentation rule

$\{X \rightarrow Y\} \models \{XZ \rightarrow YZ\}, Z \subseteq R$. Here XZ denotes $X \cup Z$

3. Transitive rule

$\{X \rightarrow Y, Y \rightarrow Z\} \models \{X \rightarrow Z\}$

4. Decomposition or Projective rule

$\{X \rightarrow YZ\} \models \{X \rightarrow Y\}$

5. Union or Additive rule

$\{X \rightarrow Y, X \rightarrow Z\} \models \{X \rightarrow YZ\}$

6. Pseudo transitive rule

$\{X \rightarrow Y, WY \rightarrow Z\} \models \{WX \rightarrow Z\}$

Armstrong's Inference Rules (2/2)

Rules 4, 5, 6 are not really necessary.

For instance, Rule 5: $\{X \rightarrow Y, X \rightarrow Z\} \models \{X \rightarrow YZ\}$ can be proved using 1, 2, 3 alone

- 1) $X \rightarrow Y$
 - 2) $X \rightarrow Z$
- } given
- 3) $X \rightarrow XY$ Augmentation rule on 1
 - 4) $XY \rightarrow ZY$ Augmentation rule on 2
 - 5) $X \rightarrow ZY$ Transitive rule on 3, 4.

Similarly, 4, 6 can be shown to be unnecessary.
But it is useful to have 4, 5, 6 as short-cut rules

Normalization:

The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations

Normal form:

Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

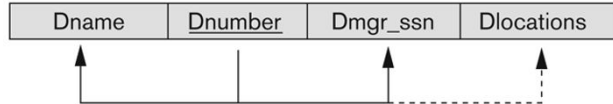
Disallows

1. composite attributes
2. multivalued attributes
3. **nested relations**; attributes whose values for an *individual tuple* are non-atomic

Normalization into 1NF

(a)

DEPARTMENT



(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 10.8

Normalization into 1NF.

(a) A relation schema that is not in 1NF. (b)

Example state of relation DEPARTMENT. (c) 1NF

version of the same

relation with redundancy.

Not in **2NF** [Because non prime attribute is not fully functional dependent on key]

In this case **Dname** can be determined by part of composite key like **Dnumber** only but the key here is **(Dname, DLocation)**.

Another Example as exercise

The following relation is not normalized because

<i>Roll no</i>	<i>Name</i>	<i>Courses taken</i>			
		<i>Course No</i>		<i>Dept</i>	<i>Sem</i>
4568	A.B Moni	CS 101		C.S.	1
		EE 545		E.E.	2
		Phy 325		Physics	1
4894	R. Chamnlal	Phy 101		Physics	1
		Chem202		Chemistry	2
		Math 103		Math.	1
		CS 101		C.S.	1
4954	R. Gupta	CS 101		C.S.	1

- (a) It is difficult to store due to non-uniform size of the attributes
- (b) Roll no. 4568 have 3 course line whereas Roll no. 4954 has only one course line
- (c) The composite attribute (CS 101, C.S., 1) is repeated
- (d) Some item lines have composite attributes

Normal Forms – 2NF

Full functional dependency:

An FD $X \rightarrow A$ for which there is no proper subset Y of X
such that $Y \rightarrow A$
(A is said to be fully functionally dependent on X)

2NF: A relation schema R is in 2NF if
every non-prime attribute is fully functionally dependent on any
key of R

**Composite
Key**

prime attribute: A attribute that is part of some key

non-prime attribute: An attribute that is not part of any key

VERY VERY important Slide

Condition for 2NF

There are **NO partial dependencies** (i.e., no attribute should depend on **only part** of a composite primary key).

However, if a table has a **single-column primary key**, then **partial dependency is not possible :)**

[SO ITS SATISFIED 2NF DIRECTLY]

IMPORTANT CASE

Can a Non-Prime Attribute Determine Another Non-Prime Attribute?

Yes, it is possible for a **non-prime attribute to determine another non-prime attribute**, but **this violates 3NF, not 2NF**.

Example:

StudentID	StudentName	Department	HOD
1	Alice	CS	Dr. X
2	Bob	IT	Dr. Y
3	Charlie	CS	Dr. X

Primary Key: StudentID

StudentName, Department, and HOD are **non-prime attributes**.

Dependency: Department $\rightarrow$ HOD (A department has a fixed Head of Department).

Here, HOD is determined by Department, which is another non-prime attribute.

This violates 3NF, not 2NF.

More Examples of **2NF**

Example

1) Book (authorName, title, authorAffiliation, ISBN, publisher, pubYear)

Keys: (authorName, title), ISBN

Not in 2NF as authorName $\rightarrow$ authorAffiliation

(authorAffiliation is not fully functionally dependent on the first key)

2) Student (rollNo, name, dept, sex, hostelName, roomNo, admitYear)

Keys: rollNo, (hostelName, roomNo)

Not in 2NF as hostelName $\rightarrow$ sex

student (rollNo, name, dept, hostelName, roomNo, admitYear)

hostelDetail (hostelName, sex)

- There are both in 2NF

Practice Questions: [click here to open the document](#)

Portion till **CIE 02 !!**

Mock Practices Link :
[click here to open responder link](#)

Normalization **3NF** and **BCNF**

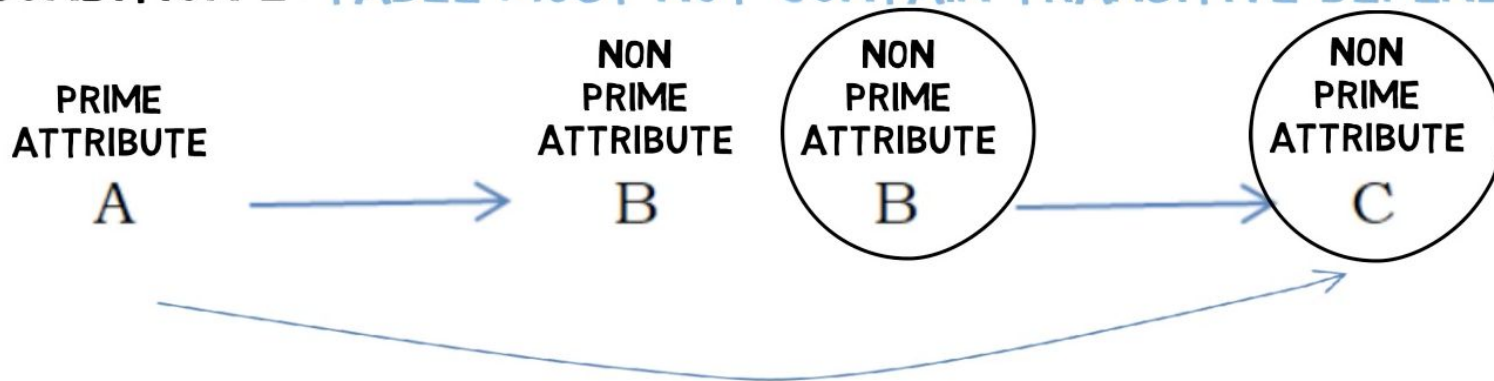
Transitive Dependencies

Transitive dependency:

An FD $X \rightarrow Y$ in a relation schema R for which there is a set of attributes $Z \subseteq R$ such that

$X \rightarrow Z$ and $Z \rightarrow Y$ and Z is not a subset of any key of R

CONDITION 2: TABLE MUST NOT CONTAIN TRANSITIVE DEPENDENCY



Transitive Dependencies

Transitive dependency:

An FD $X \rightarrow Y$ in a relation schema R for which there is a set of attributes $Z \subseteq R$ such that

$X \rightarrow Z$ and $Z \rightarrow Y$ and Z is not a subset of any key of R

Ex: student (rollNo, name, dept, hostelName, roomNo, headDept)

Keys: rollNo, (hostelName, roomNo)

rollNo $\rightarrow$ dept; dept $\rightarrow$ headDept hold

So, rollNo $\rightarrow$ headDept a transitive dependency

Head of the dept of dept D is stored redundantly in every tuple where D appears.

Relation is in 2NF but redundancy still exists.

Overview of 3NF

A relation is in the third normal form, if there is **no transitive dependency for non-prime attributes** as well as it is in the second normal form.

A relation is in 3NF if at least one of the following conditions holds in every non-trivial functional dependency $X \rightarrow Y$.

- X is a super key.
- Y is a prime attribute (each element of Y is part of some candidate key).

In other words,

A relation that is in First and Second Normal Form and in which no non-primary-key attribute is transitively dependent on the primary key, then it is in Third Normal Form (3NF).

STUD_NO	STUD_NAME	STUD_STATE	STUD_COUNTRY	STUD_AGE
1	RAM	HARYANA	INDIA	20
2	RAM	PUNJAB	INDIA	19
3	SURESH	PUNJAB	INDIA	21

FD set: { STUD_NO -> STUD_NAME,
STUD_NO -> STUD_STATE,
STUD_STATE -> STUD_COUNTRY,
STUD_NO -> STUD_AGE }

Candidate Key: {STUD_NO}

For this relation in table 4, **STUD_NO** -> **STUD_STATE** and **STUD_STATE** -> **STUD_COUNTRY** are true. So **STUD_COUNTRY** is transitively dependent on **STUD_NO**.

It violates the third normal form.

Decomposition of table

STUDENT (STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_AGE)

STATE_COUNTRY (STATE, COUNTRY)

Example 02

CAND_NO	CAND_NAME	CAND_STATE	CAND_COUNTRY	CAND_AGE
1	TINA	MAHARASHTRA	INDIA	18
2	ANJALI	RAJASTHAN	INDIA	17
3	RAHUL	RAJASTHAN	INDIA	19

Identify the above table is 3NF or not

CAND_ID	CAND_NAME	CAND_ZIP	CAND_CITY	CAND_STATE
262	Jake	201010	Noida	UP
353	Rosa	02228	Boston	US
434	Charles	60007	Chicago	US
545	Gina 0	6389	Norwich	UK
626	Terry	462007	Bhopal	MP

Identify the above table is 3NF or not

Basics practices of Functional Dependencies

Consider Relation **R(A, B, C, D, E)**

A -> BC,

C -> DE,

B -> D,

E -> A

Identify the candidate key and primary key

Understanding 3NF for R(A,B,C)

Third Normal Form (3NF) means:

1. The relation must be in 2NF (no partial dependencies)
2. No non-key attribute should depend on another non-key attribute

Example with R(A,B,C)

Let's say we have these functional dependencies:

- $A \rightarrow B$
- $B \rightarrow C$

Step 1: Find the candidate key

A determines B, and through B, it determines C

So $A \rightarrow B \rightarrow C$

Therefore, A is the candidate key

Step 2: Check for 3NF violations

$A \rightarrow B$: A is a key, so this is fine ✓

$B \rightarrow C$: B is not a key, and C is not part of any key ✗

This relation violates 3NF because we have a non-key attribute (C) depending on another non-key attribute (B) instead of depending directly on the key (A).

How to fix it

Split R into two relations:

R1(A,B) with dependency $A \rightarrow B$

R2(B,C) with dependency $B \rightarrow C$

This way, each relation will be in 3NF because all non-key attributes depend directly on the key.

Student tip

Remember: In 3NF, everything must depend on “the key, the whole key, and nothing but the key.”

Consider Relation **R(A, B, C, D, E)**

A -> BC,

CD -> E,

B -> D,

E -> A

Identify the candidate key and primary key

Consider Relation **R(A, B, C, D, E)**

A -> BC,

C -> DE,

B -> D,

E -> A

Identify the above relation is 3NF or Not ?

BCNF

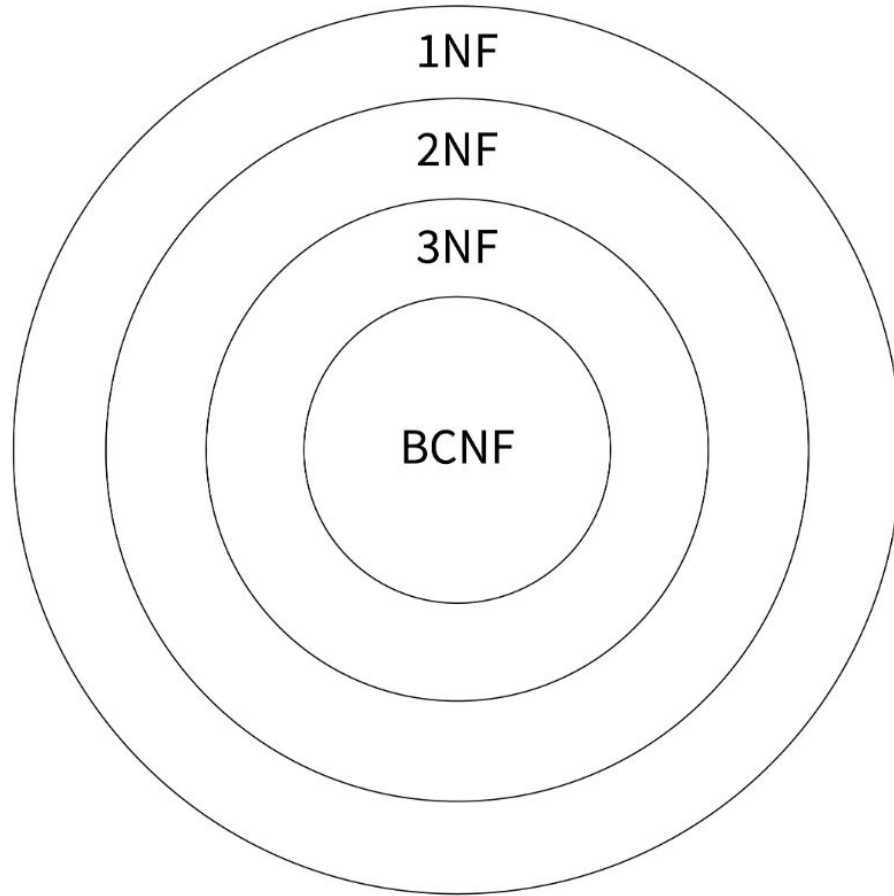
BCNF (Boyce-Codd Normal Form)

Overview of **BCNF**

A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD $X \rightarrow A$** holds in R , then **X is a superkey** of R .

OR

For any functional dependency ($A \rightarrow B$), A should be either the **super key or the candidate key**. In simple words, it means that A can't be a non-prime attribute if B is given as a prime attribute.



Each **normal form** is **strictly stronger** than the previous one

Every 2NF relation is in 1NF

Every 3NF relation is in 2NF

Every **BCNF** relation is in 3NF

Note: There exist relations that are in 3NF but not in BCNF

Simple Example

Stu_ID	Stu_Branch	Stu_Course	Branch_Number	Stu_Course_No
101	Computer Science & Engineering	DBMS	B_001	201
101	Computer Science & Engineering	Computer Networks	B_001	202
102	Electronics & Communication Engineering	VLSI Technology	B_003	401
102	Electronics & Communication Engineering	Mobile Communication	B_003	402

Stu_ID $\rightarrow$ Stu_Branch

Stu_Course $\rightarrow$ {Branch_Number, Stu_Course_No}

Identify whether the above schema is BCNF or not ?

It is not BCNF because

As we can see that neither **Stu_ID** nor **Stu_Course** is a Super Key. As the rules mentioned above clearly tell that for a table to be in BCNF, it must follow the property that for functional dependency $X \rightarrow Y$, **X must be in Super Key** and here this property fails, that's why this table is not in BCNF.

Task : Justify that the table in the next slide is
3NF not BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 10.13

A relation TEACH that
is in 3NF but not
BCNF.